

VIETNAM NATIONAL UNIVERSITY, HANOI

VNU- International School



DATA WAREHOUSING AND BUSINESS ANALYTICS
DESIGNING AND IMPLEMENTING DATA WAREHOUSE FOR DIGITAL
MEDIA STORE SALES PROCESS

Class: INS307303

Lecturer: ThS Hoàng Trọng Tiến

Group member: Lê Tuấn Sơn 21070340

Lưu Hà Linh 21070757

Hoàng Anh Đức 21070190

Phạm Thị Mỹ Dung 21070509

Phạm Thị Lan Anh 21070137

Hà Nội, January 2025

Table of Contents

I.Introduction	5
1.1. Problem Statement	5
1.2. Project Objective	5
II. Datawarehouse Design	6
2.1. Data Source	6
2.2. Dimensional Modelling.....	11
2.3. Datawarehouse Architecture	13
III. Data Warehouse Implementation.....	14
3.1. Data Source (CloudSQL)	15
3.2. Staging	16
3.3. Summary area.....	16
3.4. Data Mart (Bigquery: 2 stages)	17
3.4.1. Stage 1 - Data Extraction and Transformation	18
3.4.2. Stage 2 - Schema Design and Data Loading.....	20
3.4. SCD	21
3.5. Datahub	22
3.5.1. Create and Set up Datahub.....	22
3.5.2. Ingest data into Datahub.....	24
3.6. Composer.....	25
3.6.1. Create and Set up Composer instance.....	25
3.6.2. Automate the pipeline by using Dags.....	26
IV. BI System Implementation	30
4.1. Visualization.....	30
4.2. Machine Learning.....	34

List of Figures

Figure 1: BPMN Tracks Purchase and Transaction Update Workflow	6
Figure 2: Raw Data.....	7
Figure 3: Star Schema.....	13
Figure 4: Architecture.....	14
Figure 5: Data Warehouse implementation pipeline	14
Figure 6: Creating the PostgreSQL instance	15
Figure 7: Data in Data Source (PostgreSQL in CloudSQL).....	15
Figure 8: Creating an engine to query and extract data from PostgreSQL.....	16
Figure 9: Avro data files in the Staging area (GCS bucket).....	16
Figure 10: Creating the Dataset in BigQuery as the Summary area.....	17
Figure 11: Loading the data from GCS into BigQuery	17
Figure 12: Query Consolidating Customer Information.....	18
Figure 13: Denormalizing Track Information	19
Figure 14: Query Building the Date Dimension.....	20
Figure 15: Query creating the Fact Table.....	20
Figure 16: SCD type 2 for Inserting case	22
Figure 17: SCD type 2 for Updating case.....	22
Figure 18: SCD type 2 for Deleting case.....	22
Figure 19: Creating a new cluster.....	23
Figure 20: Helm set up	23
Figure 21: Installing Datahub	23
Figure 22: API in Kubernetes/Service.....	24
Figure 23: Datahub UI.....	24
Figure 24: Configuring Connection in Datahub	25
Figure 25: Composer Instance	25
Figure 26: Installing the necessary libraries in the Composer Environment.....	26
Figure 27: Configuring the Network in the Composer Environment.....	26
Figure 28: Function to Load the data from CloudSQL into GCS as Avro format	27
Figure 29: Function to load all the data in GCS into BiQuery	28
Figure 30: Function to create a Dimension table in BigQuery.....	28

Figure 31: Defining the Dag with schedule_interval for daily running	29
Figure 32: Function to insert new data in Data Mart.....	29
Figure 33: Overview Sheet	30
Figure 34: September revenue highlight	31
Figure 35: Revenue by each product	32
Figure 36: Revenue by Rock	33
Figure 37: The Logistic Regression model.....	34
Figure 38: Evaluation Metrics	35
Figure 39: Confusion Matrix	35
Figure 40: Churn Prediction Distribution.....	36

Table of abbreviation

Abbreviation	Meaning
GCS	Google Cloud Storage
SCD	Slowly Changing Dimension
BI	Business Intelligence
DWH	Data Warehouse
BPMN	Business Process Modeling Notation
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
OLAP	Online analytical processing

I.Introduction

1.1. Problem Statement

Businesses often face challenges in managing and analyzing large volumes of transactional data to make strategic decisions. In the digital media industry, services like Apple iTunes Store, a platform for distributing music and movies, require a robust system to understand customer behavior, optimize sales, predict churn rates, and improve customer retention strategies. Current operational data is often stored in a normalized format, making analysis inefficient. The lack of an integrated system to process, store, and visualize data hinders the ability to generate actionable insights. This project aims to address these challenges by transforming operational data into a centralized, scalable Data Warehouse (DWH) that supports efficient querying, combined with Business Intelligence (BI) tools and predictive analytics.

1.2. Project Objective

The primary goal of this project is to design and implement a comprehensive Data Warehouse (DWH) to store business data and support a Business Intelligence (BI) system for analytical purposes. The objectives include:

- Building a centralized, organized, and scalable Data Warehouse to serve as the foundation for storing business data and supporting the BI system.
- Transforming the Chinook operational dataset into a star-schema-based Data Warehouse to enable efficient querying and data retrieval.
- Supporting multi-dimensional analysis of sales, customer behavior, and transaction trends for deeper insights into business operations.
- Use machine learning to predict customer churn, understand its causes, and create strategies to reduce it.
- Facilitating insightful visualizations and reports for decision-makers to improve customer retention strategies, optimize sales performance, and enhance overall business efficiency.

II. Datawarehouse Design

2.1. Data Source

2.1.1. Business Process

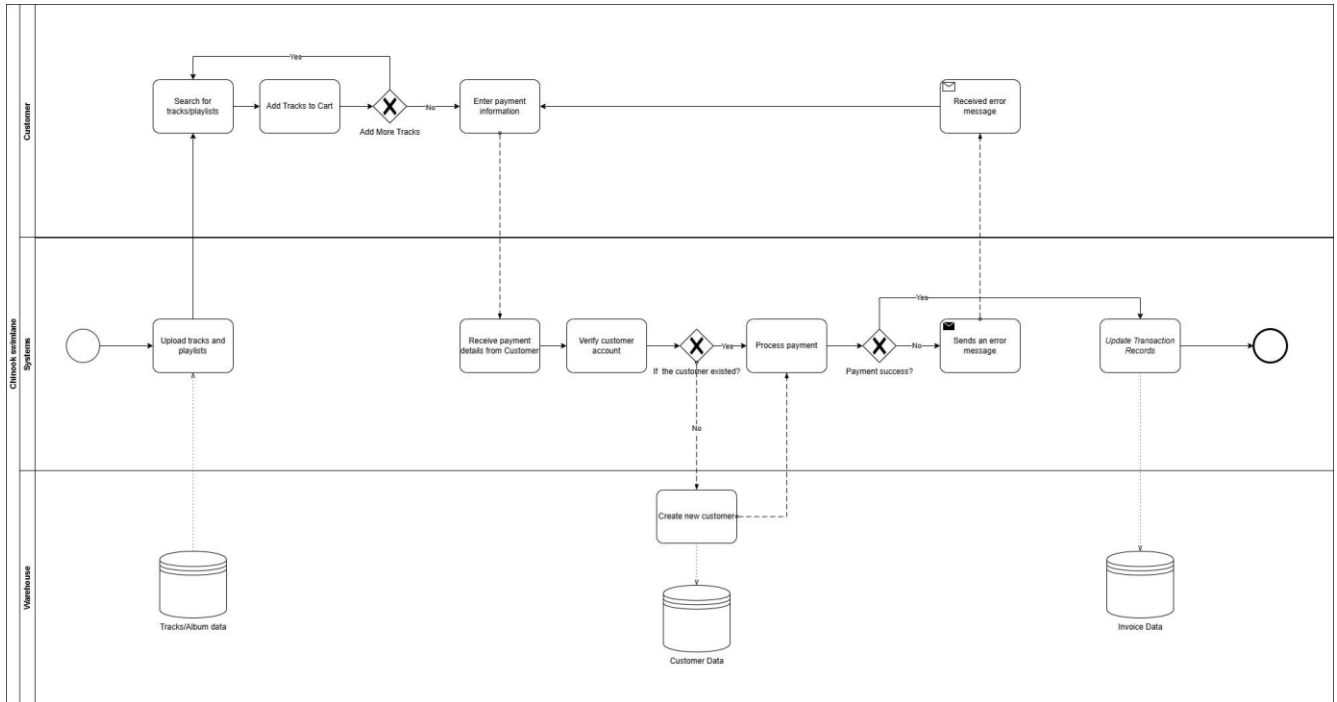


Figure 1: BPMN Tracks Purchase and Transaction Update Workflow

This BPMN diagram describes the process of purchasing tracks and playlists. It was created as a crucial step in analyzing and understanding the operational workflow. From this, our project team can determine the necessary data sources to design a Data Warehouse (DWH) in a star schema format, supporting a Business Intelligence (BI) system.

The process starts when the system uploads tracks/playlists on the website, then customers can search for music tracks or playlists they are interested in. After searching, they add the tracks to their cart. At this stage, customers can either add more tracks or proceed to the payment information step. The system receives the customer's payment information and verifies the account. If the customer already exists in the database, the system continues to process the payment. If not, the system creates a new account and stores the information in the customer database. After account verification, the system processes the payment. If the payment is successful,

the transaction information is saved into the Invoice Data database. If the payment fails, the customer receives an error notification and is asked to try again with different payment details.

After each transaction, the system updates the transaction data, including invoices, customer, and music information, to ensure the most recent data is stored in the operational database.

2.1.2. Purpose of the BPMN Diagram:

By building this BPMN diagram, our project team aims to:

- Understand the current operational processes of the system, including searching, payment, and transaction information storage.
- Identify critical data sources necessary for DWH design:
 - **Customer Data:** Information about both new and existing users.
 - **Transaction Data:** Details about invoices, payment status, and product information.
 - **Product Data:** Information about tracks, albums, or playlists.
- Provide a foundation for transforming data from the current operational database into a star schema DWH to effectively support BI analysis.

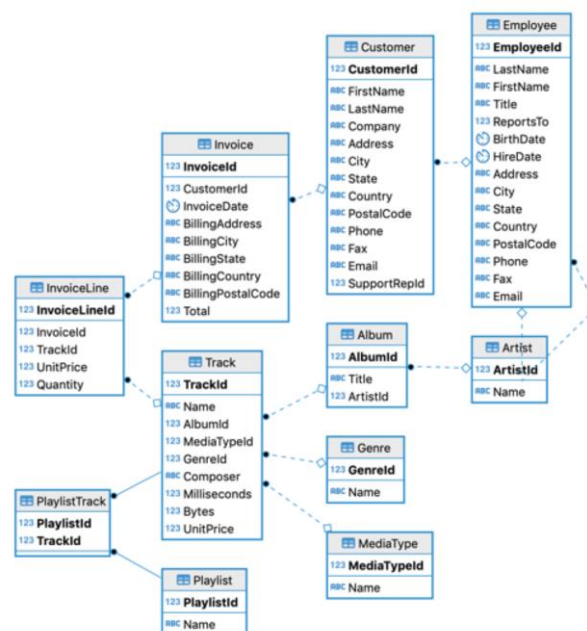


Figure 2: Raw Data

2.1.3. Explanation of Data Sources:

Album: To store information about music albums. Each album has a title and is associated with an artist.

- **AlbumId:** Primary key, unique identifier for each album.
- **Title:** Album name.
- **ArtistId:** Foreign key linked to the **Artist** table, indicating the artist or band that released the album.

Artist: To store information about artists or bands.

- **ArtistId:** Primary key, unique identifier for each artist.
- **Name:** Name of the artist or band.

Customer: To store customer information in the system.

- **CustomerId:** Primary key, unique identifier for each customer.
- **FirstName, LastName:** Customer's first and last name.
- **Company:** Customer's company name (if applicable).
- **Address, City, State, Country, PostalCode:** Contact address details.
- **Phone, Fax:** Customer's phone and fax number.
- **Email:** Customer's email address.
- **SupportRepId:** Foreign key linked to the **Employee** table, indicating the support representative assigned to this customer.

Employee: To store information about employees in the system.

- **EmployeeId:** Primary key, unique identifier for each employee.
- **FirstName, LastName:** Employee's first and last name.
- **Title:** Job title of the employee (e.g., Manager, Sales Representative).
- **ReportsTo:** Foreign key indicating the direct manager of this employee (linked to another **EmployeeId**).
- **BirthDate, HireDate:** Employee's date of birth and hire date.
- **Address, City, State, Country, PostalCode:** Contact address details.

- **Phone, Fax, Email:** Employee's contact details.

Genre: To store information about music genres.

- **GenreId:** Primary key, unique identifier for each genre.
- **Name:** Name of the genre (e.g., Rock, Pop, Jazz).

Invoice: To store transaction information for customer purchases.

- **InvoiceId:** Primary key, unique identifier for each invoice.
- **CustomerId:** Foreign key linked to the **Customer** table, indicating the buyer.
- **InvoiceDate:** Date of the invoice.
- **BillingAddress, BillingCity, BillingState, BillingCountry, BillingPostalCode:** Billing address details.
- **Total:** Total amount of the invoice.

InvoiceLine: To store detailed information about each line in an invoice, representing individual tracks purchased.

- **InvoiceLineId:** Primary key, unique identifier for each invoice line.
- **InvoiceId:** Foreign key linked to the **Invoice** table, indicating the associated invoice.
- **TrackId:** Foreign key linked to the **Track** table, indicating the purchased track.
- **UnitPrice:** Price of the track.
- **Quantity:** Quantity of the track (usually 1).

MediaType: To store information about the media format of tracks.

- **MediaTypeId:** Primary key, unique identifier for each media type.
- **Name:** Name of the media type (e.g., MPEG audio file, Protected AAC audio file).

Playlist: To store information about playlists created by customers or the system.

- **PlaylistId:** Primary key, unique identifier for each playlist.
- **Name:** Name of the playlist.

Track: To store detailed information about individual music tracks.

- **TrackId**: Primary key, unique identifier for each track.
- **Name**: Name of the track.
- **AlbumId**: Foreign key linked to the **Album** table, indicating the album containing this track.
- **MediaTypeId**: Foreign key linked to the **MediaTypes** table, indicating the track's format.
- **GenreId**: Foreign key linked to the **Genre** table, indicating the track's genre.
- **Composer**: Name of the composer.
- **Milliseconds**: Duration of the track in milliseconds.
- **Bytes**: File size of the track in bytes.
- **UnitPrice**: Price of the track.

PlaylistTrack: To store information about the relationship between playlists and tracks.

- **PlaylistId**: Foreign key linked to the **Playlist** table, indicating the playlist.
- **TrackId**: Foreign key linked to the **Track** table, indicating the track included in the playlist.

2.1.4. Checking Normalization

To ensure the database adheres to normalization principles, the first step is to confirm compliance with the First Normal Form (**1NF**). In this form, each column must contain atomic data, meaning no arrays or lists, and all tables must have a unique primary key. Additionally, there should be no repeating columns in any table.

Once **1NF** is satisfied, the database must meet the criteria for Second Normal Form (**2NF**). This requires that the database already complies with **1NF** and that there are no partial dependencies, meaning no non-key attribute should depend on a proper subset of any candidate key. For instance, in the **InvoiceLine** table, all non-key attributes (**TrackId**, **UnitPrice**, **Quantity**) depend entirely on the composite primary key (**InvoiceLineId**, **InvoiceId**).

Finally, the database must satisfy the Third Normal Form (**3NF**). This form builds upon **2NF** by ensuring there are no transitive dependencies, meaning no non-key attribute depends on another non-key attribute. For example, in the **Customer** table, attributes like **City**, **State**, and **Country** depend only on the primary key (**CustomerId**) and not on another non-key attribute such as **Address**. By adhering to these normalization rules, the database is optimized for data integrity, avoids redundancy, and minimizes anomalies during updates or deletions.

2.2. Dimensional Modelling

Step 1: Identify the Business Process

The data warehouse is designed to analyze **music sales transactions**, focusing on customer purchases, tracking details, and time dimensions.

Fact Table: Fact_Transaction is the central table in this star schema, capturing details of each sales transaction.

Dimensions:

- **Dim_Date:** Provides temporal context for analysis.
- **Dim_Track:** Details about the tracks sold, including album, genre, and media type.
- **Dim_Customer:** Information about the customers who made purchases.

Step 2: Declare the Grain

Grain: The finest level of detail captured is **each line item in a transaction**. This means one row in the Fact_Transaction table corresponds to a single track sold in a specific invoice.

Step 3: Identify the dimensions

Dim_Date: Provides context for temporal analysis such as daily, monthly, quarterly, or yearly trends. This table concludes some attributes:

- **date:** Primary key for the date dimension.
- **year, quarter_of_year, month:** For aggregations by periods.
- **day_of_week, is_weekend:** For analyzing purchasing behavior based on days.

Dim_Track: Contains details about the tracks sold. This table concludes some attributes:

- track_id: Primary key for the track.
- track_name, album_title, artist_name, genre_name, media_type_name: Provide descriptive information.
- composer, milliseconds, bytes, unit_price: Additional track details for analysis.

Dim_Customer: Stores customer details to understand customer demographics and behavior. This table concludes some attributes:

- customer_id: Primary key.
- first_name, last_name, company, phone, fax, email: Customer personal information.
- support_rep_id: Links to customer support staff for service analysis.

Step 4: Identify the Facts

Fact_Transaction: Captures quantitative measures related to sales transactions.

- **Primary Key:** Invoice_line_id
- **Foreign Keys:**
 - invoice_id: Links to Dim_Billing.
 - track_id: Links to Dim_Track.
 - customer_id: Links to Dim_Customer.
 - invoice_date: Links to Dim_Date.
- **Facts:**
 - unit_price: The price of the track.
 - quantity: Number of units purchased.
 - created_at: Timestamp of the transaction.

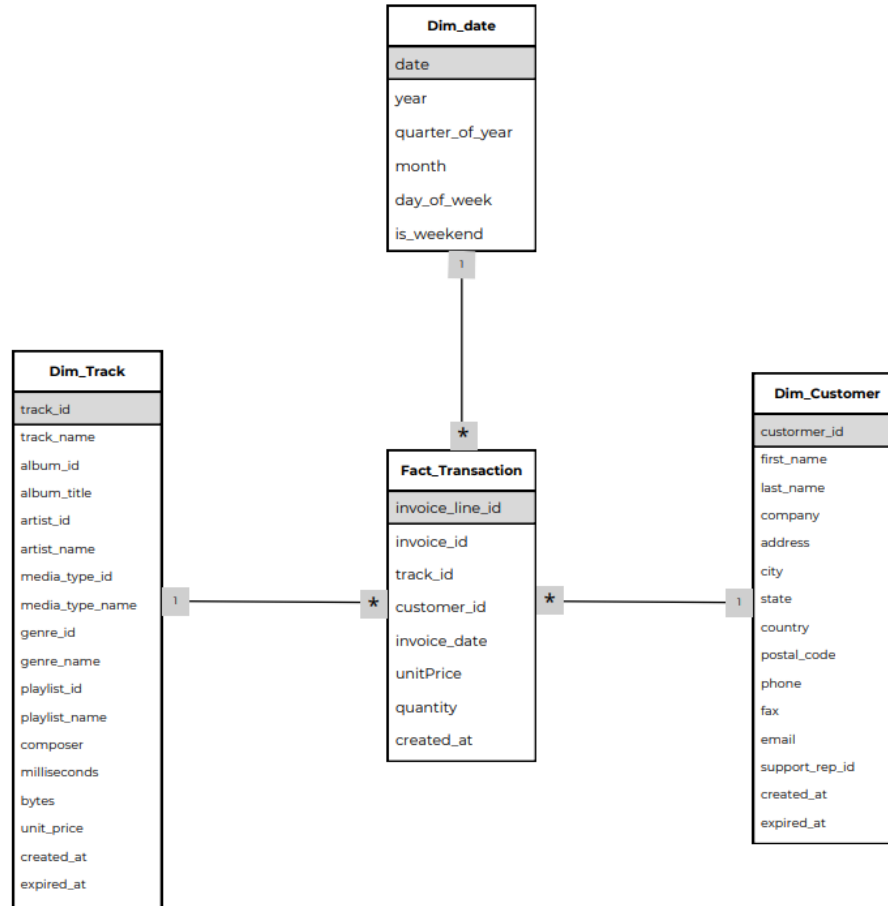


Figure 3: Star Schema

2.3. Datawarehouse Architecture

The proposed data warehouse architecture is designed to systematically process and analyze the Chinook dataset obtained from the Apple iTunes library. This architecture's primary data sources include data for artists, albums, media tracks, invoices, and customers. The Extract, Transform, Load (ETL) process ensures data integrity by extracting relevant information, transforming it into a consistent format, and loading it into the data warehouse. Dedicated data warehouses are created for both customer and performance data, allowing for focused analysis. Multidimensional Online Analytical Processing (OLAP) blocks are deployed to facilitate efficient and in-depth analysis, providing decision-makers with a comprehensive view of the factors influencing customers.

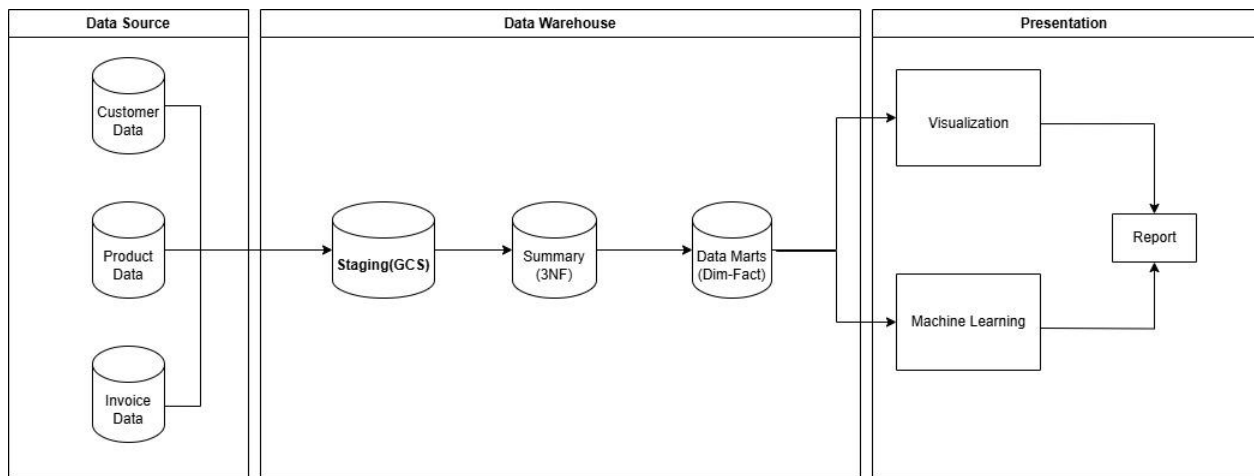


Figure 4: Architecture

Basically, the pipeline of the Data Warehouse starts at the Data Source, then data is extracted into the Staging area and then loaded into the Summary area then transformed into the Data Mart which allows users to analyze efficiently.

III. Data Warehouse Implementation

In order to implement our Data Warehouse, we followed this pipeline which starts from the Data Source imported into PostgreSQL in Cloud SQL. Then we extracted the data to the Staging area which is the Cloud Storage Bucket. Following that, we loaded the data into the Summary area in BigQuery and transformed the data from 3NF into Data Mart as Star Schema (Dim-fact tables).

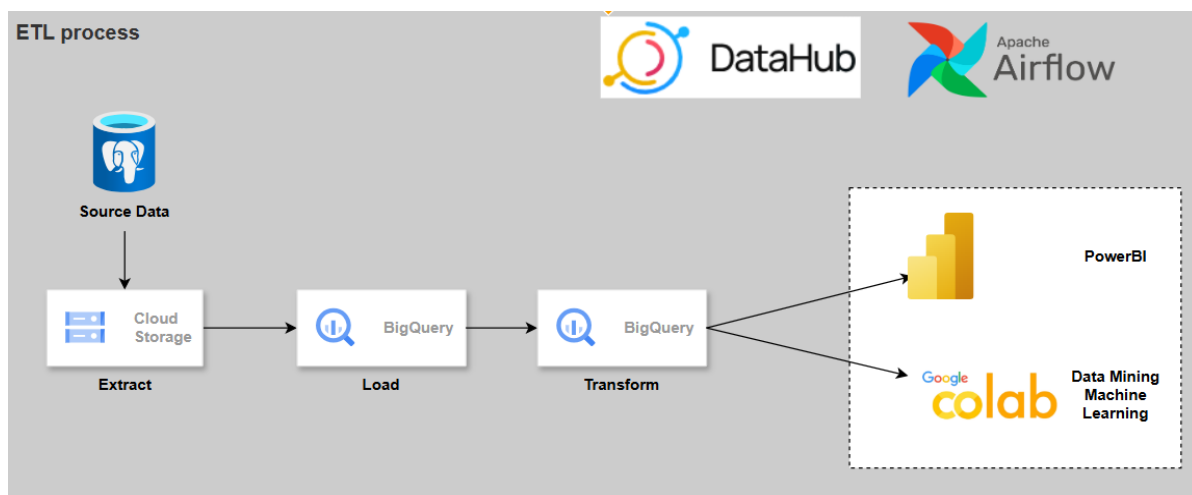


Figure 5: Data Warehouse implementation pipeline

Finally, the data in the Data mart is used for further analytics as visualization using PowerBI and Machine Learning Implementation using Colab. All the pipeline is automated by using Airflow and all the metadata are observed by Datahub.

3.1. Data Source (CloudSQL)

In this project, we use Cloud SQL to create a PostgreSQL as our Data Source. In Google Cloud SQL, we created a new PostgreSQL instance.

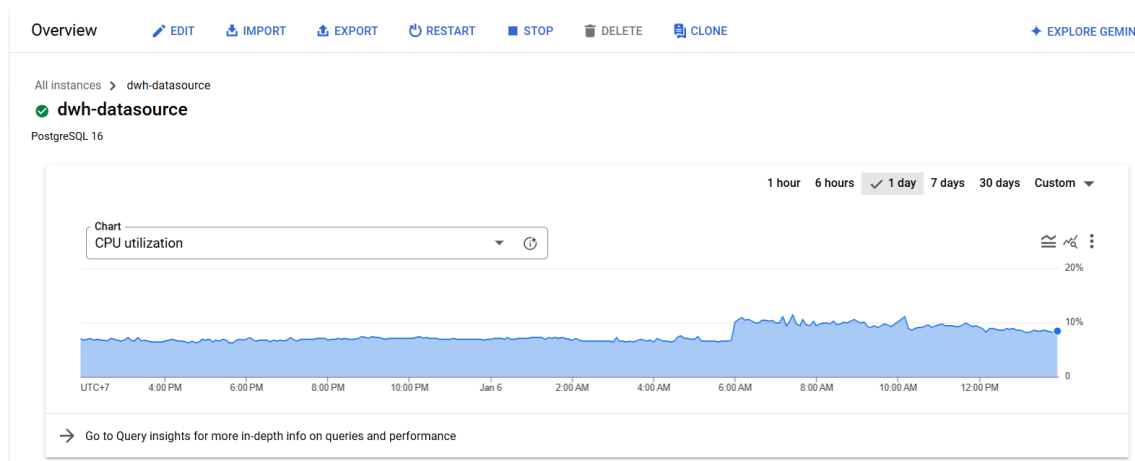


Figure 6: Creating the PostgreSQL instance

Then we imported the Chinook Dataset into the Postgres Database by importing the Chinook SQL file.

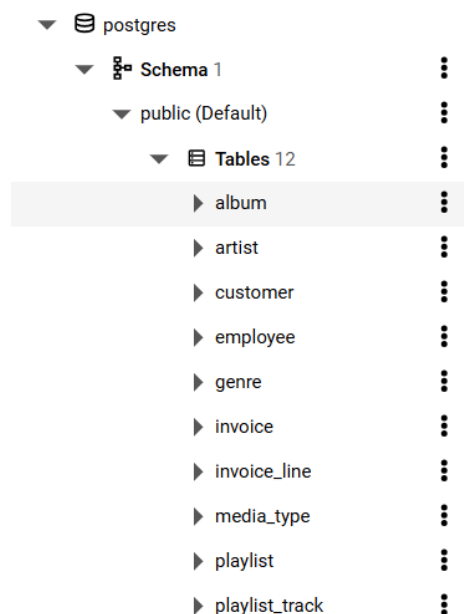


Figure 7: Data in Data Source (PostgreSQL in CloudSQL)

3.2. Staging

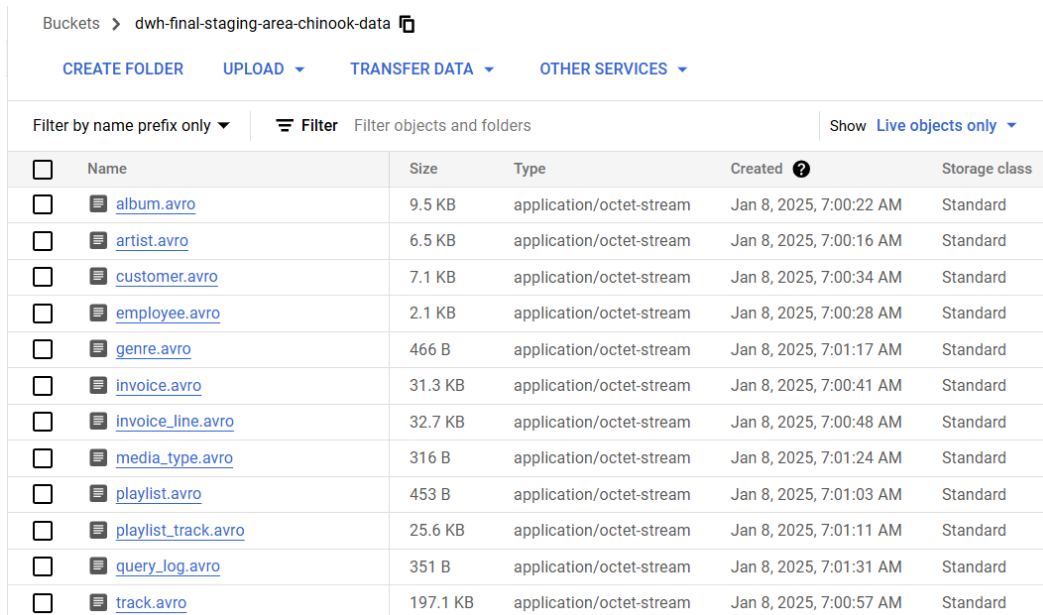
Then, we extract the data from the Data Source into the Staging area, Google Cloud Storage Bucket.

Firstly, we created an SQL engine to query in Postgres in CloudSQL through Python to get the table's names and the data.

```
def get_postgresql_connection():
    postgres_conn = {
        "user": "postgres",
        "password": "1683",
        "host": "172.30.112.3",
        "port": "5432",
        "database": "postgres"
    }
    return create_engine(f"postgresql://{postgres_conn['user']}:{postgres_conn['password']}@"
                        f"{postgres_conn['host']}:{postgres_conn['port']}/{postgres_conn['database']}")
```

Figure 8: Creating an engine to query and extract data from PostgreSQL

Furthermore, we created a bucket in GCS as the staging area, and then we transformed and extracted the data in Avro format and pushed them into the GCS bucket.



The screenshot shows the Google Cloud Storage interface for a bucket named 'dwh-final-staging-area-chinook-data'. It displays a list of 13 Avro files, each with a checkbox, name, size, type, creation time, and storage class. The files are: album.avro, artist.avro, customer.avro, employee.avro, genre.avro, invoice.avro, invoice_line.avro, media_type.avro, playlist.avro, playlist_track.avro, query_log.avro, and track.avro.

☐	Name	Size	Type	Created	Storage class
☐	album.avro	9.5 KB	application/octet-stream	Jan 8, 2025, 7:00:22 AM	Standard
☐	artist.avro	6.5 KB	application/octet-stream	Jan 8, 2025, 7:00:16 AM	Standard
☐	customer.avro	7.1 KB	application/octet-stream	Jan 8, 2025, 7:00:34 AM	Standard
☐	employee.avro	2.1 KB	application/octet-stream	Jan 8, 2025, 7:00:28 AM	Standard
☐	genre.avro	466 B	application/octet-stream	Jan 8, 2025, 7:01:17 AM	Standard
☐	invoice.avro	31.3 KB	application/octet-stream	Jan 8, 2025, 7:00:41 AM	Standard
☐	invoice_line.avro	32.7 KB	application/octet-stream	Jan 8, 2025, 7:00:48 AM	Standard
☐	media_type.avro	316 B	application/octet-stream	Jan 8, 2025, 7:01:24 AM	Standard
☐	playlist.avro	453 B	application/octet-stream	Jan 8, 2025, 7:01:03 AM	Standard
☐	playlist_track.avro	25.6 KB	application/octet-stream	Jan 8, 2025, 7:01:11 AM	Standard
☐	query_log.avro	351 B	application/octet-stream	Jan 8, 2025, 7:01:31 AM	Standard
☐	track.avro	197.1 KB	application/octet-stream	Jan 8, 2025, 7:00:57 AM	Standard

Figure 9: Avro data files in the Staging area (GCS bucket)

3.3. Summary area

After storing the data in the staging area, we loaded the data into the summary area (BigQuery).

First, we created a Dataset in BigQuery.

```

def create_bigquery_dataset_if_not_exists(project_id, dataset_name):
    """Create a BigQuery dataset if it does not exist."""
    client = bigquery.Client()
    dataset_id = f"{project_id}.{dataset_name}"

    try:
        client.get_dataset(dataset_id) # Check if dataset exists
        print(f"Dataset {dataset_id} already exists.")
    except NotFound:
        # Create the dataset
        dataset = bigquery.Dataset(dataset_id)
        dataset.location = "US" # Specify your dataset location
        client.create_dataset(dataset, exists_ok=True)
        print(f"Created dataset {dataset_id}.")

```

Figure 10: Creating the Dataset in BigQuery as the Summary area

Then, we performed the load process by using GCSToBigQueryOperator to load all the tables in GCS into BigQuery

```

for table_name in list_files:
    load_avro_to_bigquery = GCSToBigQueryOperator(
        task_id=f"load_{table_name}_to_bigquery",
        bucket=BUCKET_NAME,
        source_objects=[f"{table_name}.avro"],
        destination_project_dataset_table=f"{PROJECT_ID}.{DATASET_NAME}.{table_name}",
        source_format='AVRO',
        write_disposition='WRITE_TRUNCATE',
        autodetect=True,
    )

    # Ensure this task runs after the dataset creation check
    previous_task >> load_avro_to_bigquery
    previous_task = load_avro_to_bigquery

```

Figure 11: Loading the data from GCS into BigQuery

3.4. Data Mart (Bigquery: 2 stages)

The implementation of the Data Mart in BigQuery involved two main stages: Data Extraction and Transformation from the Summary Area (Stage 1) and Schema Design and Loading into the Data Mart (Stage 2). The objective was to convert the normalized schema in the summary area into a dimensional star schema optimized for analytical queries and reporting.

3.4.1. Stage 1 - Data Extraction and Transformation

In this stage, data from the summary are extracted and transformed into a format suitable for a dimensional Data mart. The transformation process included denormalization and the creation of dimensional and fact tables.

Step 1: Consolidating Customer Information

- Source: customer table in the summary area.
- Transformation: We selected relevant attributes for customer profiling

```
CREATE TABLE `high-diagram-443713-k1.Data_mart.dim_customer` AS
SELECT DISTINCT
  customer_id,
  first_name,
  last_name,
  company,
  address,
  city,
  state,
  country,
  postal_code,
  phone,
  fax,
  email,
  support_rep_id,
  CURRENT_TIMESTAMP() AS created_at,
  NULL AS expired_at
FROM `high-diagram-443713-k1.summary_area.customer`;
```

Figure 12: Query Consolidating Customer Information

Step 2: Denormalizing Track Information

- Source: track, album, artist, genre, media_type, and playlist_track tables in the summary area.
- Transformation: We combined track details with album, artist, genre, and media type information and aggregated playlist names into a single column using STRING_AGG.

```

CREATE TABLE `high-diagram-443713-k1.Data_mart.dim_track` AS
SELECT
  t.track_id,
  t.name AS track_name,
  t.composer,
  t.milliseconds,
  t.bytes,
  t.unit_price,
  a.album_id,
  a.title AS album_title,
  art.artist_id,
  art.name AS artist_name,
  g.genre_id,
  g.name AS genre_name,
  m.media_type_id,
  m.name AS media_type_name,
  STRING_AGG(p.name, ', ') AS playlists, -- Gộp danh sách tên playlist
  CURRENT_TIMESTAMP() AS created_at, -- Ngày tạo
  NULL AS expired_at
FROM
  `high-diagram-443713-k1.summary_area.track` AS t
LEFT JOIN
  `high-diagram-443713-k1.summary_area.album` AS a
  ON t.album_id = a.album_id
LEFT JOIN
  `high-diagram-443713-k1.summary_area.artist` AS art
  ON a.artist_id = art.artist_id
LEFT JOIN
  `high-diagram-443713-k1.summary_area.genre` AS g
  ON t.genre_id = g.genre_id
LEFT JOIN
  `high-diagram-443713-k1.summary_area.media_type` AS m
  ON t.media_type_id = m.media_type_id
LEFT JOIN
  `high-diagram-443713-k1.summary_area.playlist_track` AS pt
  ON t.track_id = pt.track_id
LEFT JOIN
  `high-diagram-443713-k1.summary_area.playlist` AS p
  ON pt.playlist_id = p.playlist_id
GROUP BY
  t.track_id, t.name, t.composer, t.milliseconds, t.bytes, t.unit_price,
  a.album_id, a.title, art.artist_id, art.name, g.genre_id, g.name,
  m.media_type_id, m.name;

```

Figure 13: Denormalizing Track Information

Step 3: Building the Date Dimension

- Source: invoice_date from the fact_transaction table.
- Transformation: We got the min invoice_date from the fact_transaction table and got the current date, then created the daily date. Then, We extracted year, month, quarter, and day attributes. Then, added derived columns like day_of_week (such as Monday, Tuesday,..) and is_weekend, Which indicates whether that day is a weekend or not (using Case When). This dim_date will be updated daily.

```
--Thêm bảng dim_date
CREATE TABLE Data_mart.dim_date AS
WITH date_range AS (
    SELECT
        MIN(invoice_date) AS start_date,
        CURRENT_DATE() AS end_date
    FROM Data_mart.fact_transaction
),
all_dates AS (
    SELECT
        DATE_ADD(start_date, INTERVAL n DAY) AS date
    FROM date_range
    CROSS JOIN UNNEST(GENERATE_ARRAY(0, DATE_DIFF(end_date, start_date, DAY))) AS n
)
SELECT
    date,
    EXTRACT(YEAR FROM date) AS year,
    EXTRACT(QUARTER FROM date) AS quarter_of_year,
    EXTRACT(MONTH FROM date) AS month,
    EXTRACT(DAYOFWEEK FROM date) AS day_of_week,
    CASE WHEN EXTRACT(DAYOFWEEK FROM date) IN (1, 7) THEN TRUE ELSE FALSE END AS is_weekend
FROM all_dates;
```

Figure 14: Query Building the Date Dimension

Step 4: Creating the Fact Table

- Source: invoice and invoice_line tables in the summary area.
- Transformation: We linked transactional details with customer and invoice data and focused on measures (unit_price, quantity) and foreign keys (invoice_id, track_id).

```
CREATE TABLE `high-diagram-443713-k1.Data_mart.fact_transaction` AS
SELECT
    il.invoice_line_id,
    il.track_id,
    il.unit_price,
    il.quantity,
    i.invoice_id,
    i.customer_id,
    DATE(TIMESTAMP_MILLIS(i.invoice_date)) AS invoice_date,
    CURRENT_TIMESTAMP() AS created_at
FROM `high-diagram-443713-k1.summary_area.invoice_line` il
JOIN `high-diagram-443713-k1.summary_area.invoice` i
ON il.invoice_id = i.invoice_id;
```

Figure 15: Query creating the Fact Table

3.4.2. Stage 2 - Schema Design and Data Loading

After transforming the data, it was undertaken to load and establish the star schema

Schema Structure:

- Fact Table: fact_transaction contains measures and references to dimension tables.
- Dimension Tables: dim_billing, dim_customer, dim_track, and dim_date hold denormalized details for efficient querying.

Data Loading:

- Tables were created in the DWH dataset of the BigQuery project.
- Data was loaded using INSERT INTO statements for small transformations or direct CREATE TABLE AS SELECT for larger transformations.

Optimization:

- Partitioned fact_transaction on invoice_date for performance.
- Clustered dim_track by genre_id and album_id to speed up queries filtering on these attributes.

3.4. SCD

In this project, we performed the Slowly Changing Dimension type 2. Our Data Mart has two-dimensional tables dim_track and dim_customer derived from the track, album, artist, customer, genre, media_type, playlist, and playlist_track tables.

Then, we only observed the changes in the above table in the data source. We observed the changes in 3 cases: Insert, Update, and Delete data.

Firstly, we generated fields created_at, which is set as the current timestamp, and expired_at, which is set as Null in a two-dimensional table.

In the Insert case, when a new data row is added to the table, we insert a new row with a new unique ID and other fields and set the created_at as the current timestamp and expired_at as Null.

Row	customer_id	first_name	created_at	expired_at
1	60	TuanSon	2025-01-08 16:16:52.913612 U...	null
2	24	Frank	2025-01-08 16:16:52.913612 U...	null
3	19	Tim	2025-01-08 16:16:52.913612 U...	null
4	18	Michelle	2025-01-08 16:16:52.913612 U...	null
5	58	Manoj	2025-01-08 16:16:52.913612 U...	null
6	59	Puja	2025-01-08 16:16:52.913612 U...	null

Figure 16: SCD type 2 for Inserting case

In the case of Updating data, we created a new row of the data updated with the modified fields but with the same ID, set the created_at field to the current timestamp, which helps us identify the updated data, and set the expired_at field to Null.

Row	customer_id	first_name	created_at	expired_at
1	60	Son	2025-01-08 16:27:57.089282 U...	null
2	60	TuanSon	2025-01-08 16:16:52.913612 U...	null
3	24	Frank	2025-01-08 16:16:52.913612 U...	null
4	19	Tim	2025-01-08 16:16:52.913612 U...	null
5	18	Michelle	2025-01-08 16:16:52.913612 U...	null
6	58	Manoj	2025-01-08 16:16:52.913612 U...	null
7	59	Puja	2025-01-08 16:16:52.913612 U...	null

Figure 17: SCD type 2 for Updating case

Finally, in the Delete case, if we want to delete a row, we'll set the expired_at as the Unix current timestamp which helps us to identify and filter the deleted row.

Row	customer_id	first_name	created_at	expired_at
1	60	TuanSon	2025-01-08 16:51:22.470809 U...	1736355326
2	19	Tim	2025-01-08 16:51:22.470809 U...	null
3	18	Michelle	2025-01-08 16:51:22.470809 U...	null
4	24	Frank	2025-01-08 16:51:22.470809 U...	null
5	59	Puja	2025-01-08 16:51:22.470809 U...	null

Figure 18: SCD type 2 for Deleting case

3.5. Datahub

3.5.1. Create and Set up Datahub

In our project, we implemented Datahub as our metadata management platform. We installed Datahub GCP Google Kubernetes Engine (GKE) .

Firstly, we created a new cluster using Google Cloud Shell

```
lts_oc_1682003@cloudshell:~$ cloud container clusters create dwh-cluster \
  --zone asia-southeast1-c \
  --disk-size 15GB
```

Figure 19: Creating a new cluster

After our cluster was set up successfully, we set up Dathub with Helm

We performed some setup for Helm

```
-- Add datahub helm repo
helm repo add datahub https://helm.datahubproject.io/
-- Deploy dependencies
Helm install prerequisites datahub/datahub-prerequisites|
```

Figure 20: Helm set up

After that, we install Datahub through Helm.

```
lts_oc_1682003@cloudshell:~$ helm install datahub datahub/datahub
```

Figure 21: Installing Datahub

After the installation, we can access the Datahub UI through the frontend API in Kubernetes/Service.

Filter Is system object : False Filter services and ingresses								
☐	Name ↑	Status	Type	Endpoints	Pods	Namespace	Clusters	
☐	datahub-acryl-datahub-actions	✓ OK	Cluster IP	34.118.237.186	1/1	default	datawarehouse	
☐	datahub-datahub-frontend	✓ OK	External load balancer	34.126.65.196:9002	1/1	default	datawarehouse	▼
☐	datahub-datahub-gms	✓ OK	External load balancer	34.143.230.198:8080	1/1	default	datawarehouse	▼
☐	elasticsearch-master	✓ OK	Cluster IP	34.118.239.185	1/1	default	datawarehouse	
☐	elasticsearch-master-headless	✓ OK	Cluster IP	None	1/1	default	datawarehouse	
☐	prerequisites-kafka	✓ OK	Cluster IP	34.118.227.198	1/1	default	datawarehouse	
☐	prerequisites-kafka-broker-headless	✓ OK	Cluster IP	None	1/1	default	datawarehouse	
☐	prerequisites-mysql	✓ OK	Cluster IP	34.118.234.233	1/1	default	datawarehouse	
☐	prerequisites-mysql-headless	✓ OK	Cluster IP	None	1/1	default	datawarehouse	

Figure 22: API in Kubernetes/Service.

3.5.2. Ingest data into Datahub

In this project, we ingested the data in BigQuery, which is our Data Warehouse into Datahub for metadata management.

We ingested data through the DataHub frontend

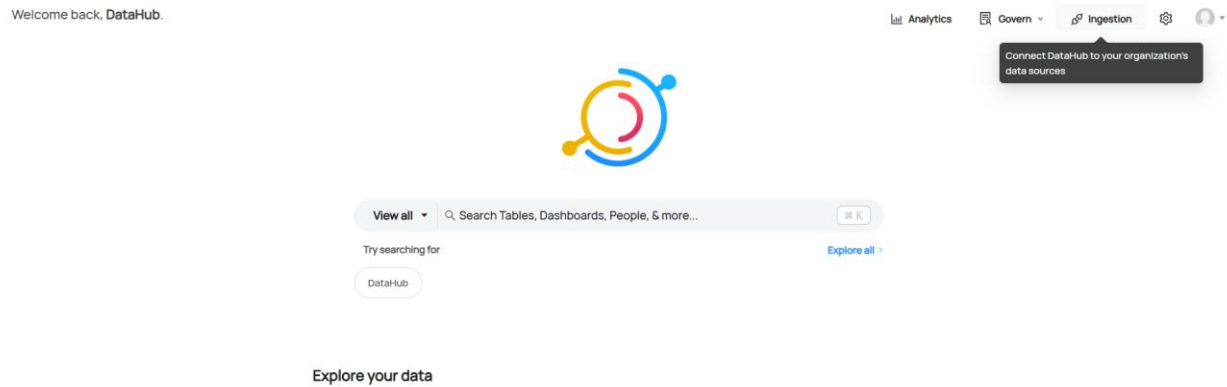


Figure 23: Datahub UI

To perform the ingestion, we configured the connection of Bigquery - Datahub using a Service account which is assigned the necessary roles

Choose Data Source — **2 Configure Connection** — 3 Sync Schedule — 4 Finish up

Let's get connected!

To import from BigQuery, we'll need some more information to connect to your instance.

Check out the [BigQuery Guide](#) to understand the prerequisites, learn about available settings, and view examples to help connect to the data source.

BigQuery Details Form YAML

Connection

* **Project ID** Secret Field
my-project-123

* **Private Key** Secret Field
---BEGIN PRIVATE KEY---...---END PRIVATE KEY---

* **Private Key Id** Secret Field
d0121d0000882411234e11166c6aaa23ed5d74e0

* **Client Email** Secret Field
client_email@gmail.com

* **Client ID** Secret Field
123456789098765432101

Figure 24: Configuring Connection in Datahub

3.6. Composer

To automate all the stages of the pipeline to implement the Data Warehouse as above, we used Composer (Airflow). By creating Dags to create and schedule tasks, we performed all tasks like extracting data from Source into Staging, loading the data in Staging into the Summary area, and transforming data into the Dimensional Model in Data Mart. Furthermore, we scheduled the ELT pipeline to run daily and also dealt with the Slowly Changing Dimension.

3.6.1. Create and Set up Composer instance

Firstly, we created a new Composer Instance in Google Cloud. After creating, we had an instance like this, we also can access the Airflow UI through the Airflow webserver API.

Composer

Environments

CREATE

REFRESH

DELETE

Filter

Filter environments

State	Name	Location	Composer version	Airflow version	Creation time	Update time	Airflow webserver	DAG list	Logs	DAGs folder	Labels
☒	final-dwh	asia-southeast1	3	2.10.2-build.4	1/7/25, 9:08 PM	1/8/25, 10:57 AM	Airflow	DAGs	Logs	DAGs	None

Figure 25: Composer Instance

Then, we installed the necessary libraries in the PYPI packages of Composer Environment details.

✓ **final-dwh** This environment is running

< G LOGS DAGS ENVIRONMENT CONFIGURATION

[EDIT](#)

Required libraries from the Python Package Index (PyPI)

Name	Version
apache-airflow-providers-google	-
apache-airflow-providers-postgres	-
psycopg2-binary	-
apache-airflow-providers-http	-
google-cloud-storage	-
sqlalchemy	-
avro-python3	-

Figure 26: Installing the necessary libraries in the Composer Environment

Moreover, we also need to configure the network in the Composer Environment. We set the Network attachment as the Default connection in the VPC network, by setting up this, we allowed Composer to use the VPC network to connect with CloudSQL, BigQuery, ...

Network configuration			
Network attachment	projects/high-diagram-443713-k1/regions/asia-southeast1/networkAttachments/asia-southeast1-final-dwh-153a5648-d971-4105-9fc1-d7c725b82e	EDIT	
Network ID	projects/high-diagram-443713-k1/global/networks/default		
Subnetwork ID	projects/high-diagram-443713-k1/regions/asia-southeast1/subnetworks/default		
Network tags	None		
Networking type	Public IP EDIT		
Web server access control	All IP addresses have access (default) EDIT		
Access to public PyPI package repositories	Allow installation of packages from public internet repositories (default) EDIT		

Figure 27: Configuring the Network in the Composer Environment

3.6.2. Automate the pipeline by using Dags

We wrote Dags to allow the ELT pipeline to run automatically. For the first time, we loaded all the historical data from the Data source into GCS, extracted data from GCS into BigQuery, and transformed the data in BigQuery into Dim-fact tables. All the Dags codes are provided in the Appendix.

- Extract Process:

For the Extract Dags, we first created and Postgres connection to query and extract data from Postgres. After creating a new bucket in GCS if it does not exist, we load the data from Postgres into the created GCS bucket in AVRO format, which requires some transforming, processing steps. (Detail in Appendix 1)

```
def extract_and_upload_to_gcs():
    logger.info("Fetching table names from PostgreSQL...")
    table_names = fetch_table_names()

    dtype_to_avro = {
        'object': 'string',
        'int64': 'long',
        'float64': 'double',
        'datetime64[ns]': {
            "type": "long",
            "logicalType": "timestamp-micros"
        },
        'bool': 'boolean',
        'category': 'string',
    }

    for table_name in table_names:
        logger.info(f"Processing table {table_name}")
        engine = get_postgresql_connection()
        with engine.connect() as conn:
            query = f'SELECT * FROM "{table_name}"'
```

Figure 28: Function to Load the data from CloudSQL into GCS as Avro format

- Load Process:

Secondly, for the Load Dags, after creating a function to create a new dataset as a summary area, we used the GCSToBigQueryOperator() function to load all the tables in GCS (Staging area) into BigQuery. (Detail Dags in the Appendix 1)

```

for table_name in list_files:
    load_avro_to_bigquery = GCSToBigQueryOperator(
        task_id=f"load_{table_name}_to_bigquery",
        bucket=BUCKET_NAME,
        source_objects=[f"{table_name}.avro"],
        destination_project_dataset_table=f"{PROJECT_ID}.{DATASET_NAME}.{table_name}",
        source_format='AVRO',
        write_disposition='WRITE_TRUNCATE',
        autodetect=True,
    )

    # Ensure this task runs after the dataset creation check
    previous_task >> load_avro_to_bigquery
    previous_task = load_avro_to_bigquery

```

Figure 29: Function to load all the data in GCS into BiQuery

- Transform Process:

In the transform process, we transform the data in the summary area from 3NF into a Dimensional Model (Dim-fact tables). By using the `BigQueryInsertJobOperator()` function, we can run the above transform query in the Data Mart implementation in BigQuery automatically.

```

create_dim_customer = BigQueryInsertJobOperator(
    task_id='create_dim_customer',
    configuration={
        "query": {
            "query": """
CREATE TABLE `high-diagram-443713-k1.Data_mart.dim_customer` AS
SELECT DISTINCT
    customer_id,
    first_name,
    last_name,
    company,
    address,
    city,
    state,
    country,
    postal_code,
    phone,
    fax,
    email,
    support_rep_id,
    CURRENT_TIMESTAMP() AS created_at,
    NULL AS expired_at
FROM `high-diagram-443713-k1.summary_area.customer`;
            """
        }
    }
)

```

Figure 30: Function to create a Dimension table in BigQuery

- Daily ELT process:

After loading all the historical data, we set up another pipeline to perform the ELT process daily, by setting up the `schedule_interval` as daily.

First, we set up a Dag to perform extract and load daily. Then we created another Dag to perform the transform process which ran after the previous Dag.

```
with DAG(
    dag_id = 'extract_load_dag',
    default_args=default_args,
    description='Extract data from PostgreSQL to Google Cloud Storage, then load to BigQuery',
    start_date=datetime(2025, 1, 1),
    schedule_interval='0 8 * * *',
    catchup=False,
    tags=['extract', 'load'],
```

Figure 31: Defining the Dag with `schedule_interval` for daily running

Then, we ran a Dag to create `dim_date` daily, which created a `dim_date` table with all day between the earliest `invoice_date` in the fact table and the current date.

- Slowly Changing Dimension

In this project, we also perform the SCD type 2, which can be processed by running Dag. We handled all the cases like inserting, updating, and deleting data in the source. The rules for modifying the fact table in SCD type 2 are the same as above mentioned, and the change in the source can be automatically streamed into the Data Warehouse by using Dag which captures the changes in dimension daily.

```
def append_to_dim_tables(**kwargs):
    append_queries = [
        # Append into dim_track
        f"""
        INSERT INTO `{PROJECT_ID}`.{DWH_DATASET}.dim_track`
        SELECT
            t.track_id,
            t.name AS track_name,
            t.composer,
            t.milliseconds,
            t.bytes,
            t.unit_price,
            a.album_id,
            a.title AS album_title,
            art.artist_id,
            art.name AS artist_name,
            g.genre_id,
            g.name AS genre_name,
            m.media_type_id,
            m.name AS media_type_name,
            COALESCE(STRING_AGG(n.name, ' '), '') AS playlists.
```

Figure 32: Function to insert new data in Data Mart

IV. BI System Implementation

4.1. Visualization



Figure 33: Overview Sheet

Our dashboard provides an overview of business performance, highlighting both positive aspects. The dashboard provides a comprehensive overview of business performance, highlighting both positive aspects and challenges. While the number of customers and transactions remained stable throughout the year, there was a noticeable decline of 14% in both metrics compared to the previous month and the same period last year. This trend could signal potential issues with customer acquisition and retention. Despite this, revenue only saw a slight decrease of 2% month-over-month but showed a significant 24% increase year-over-year, driven by a notable rise in the average order value (AOV), which increased by 14% from the previous month and 45% from the previous year. This indicates that while fewer customers were making purchases, those who did were spending more per transaction.

In the first half of the year, revenue primarily came from Rock and Metal products, which are generally low-value items often released in albums with few tracks. As a result, despite steady transaction volumes, overall revenue did not see significant growth. However, a

substantial revenue surge occurred in August, September, and October, driven by the introduction of TV Shows as a new category. These products, typically released with multiple tracks and higher price points, contributed significantly to the revenue boost. In September 2024, a notable spike in revenue was recorded, with TV Shows accounting for 56.12% of total revenue, possibly due to a special sales campaign or event.

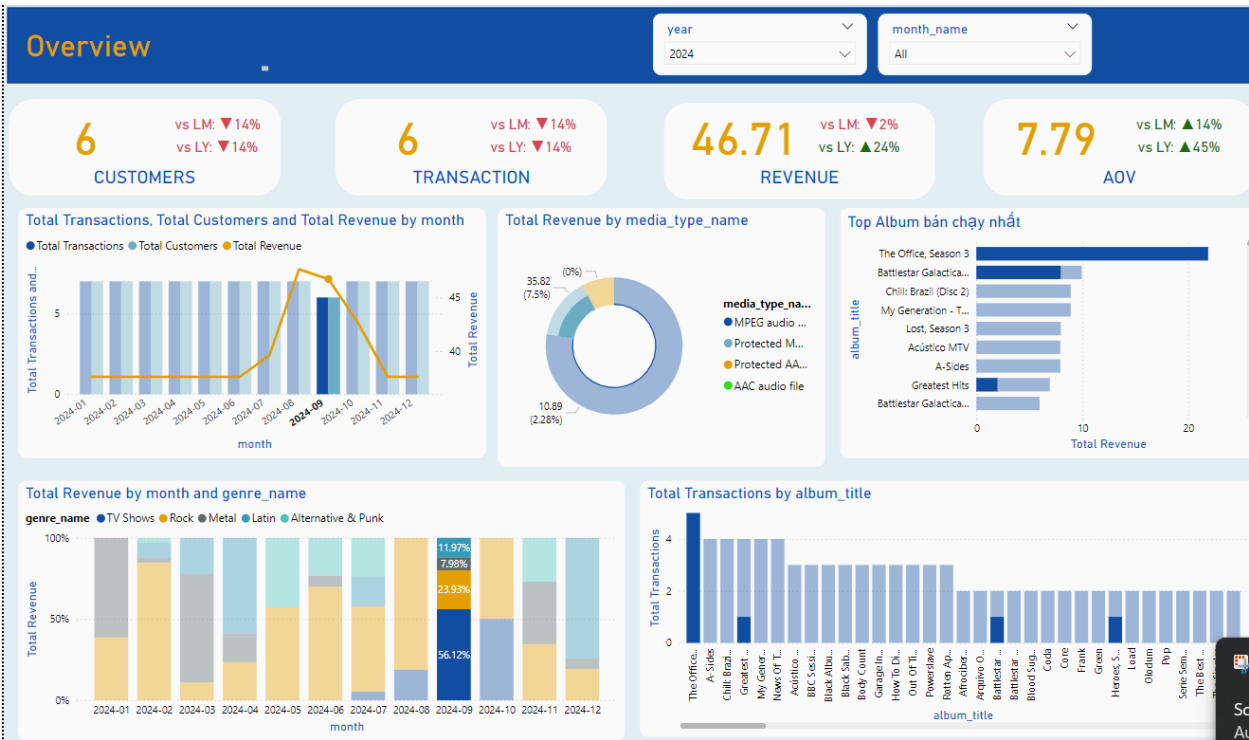


Figure 34: September revenue highlight

Towards the end of the year, revenue saw a decline as the TV Show trend began to wane. During this period, there was a resurgence of Alternative & Punk and Latin genres. Despite steady transaction numbers, these genres could not sustain the higher revenue levels due to their lower price points. The best-selling products included "The Office, Season 3" and "Battlestar Galactica," emphasizing the significant role of TV shows and music albums in revenue generation.

In summary, the combination of fewer customers and transactions, offset by higher spending per transaction, and the strategic focus on high-value TV Show products during peak months, helped drive overall revenue growth. However, the shift back to lower-value genres in the latter part of the year led to a noticeable decline in revenue.

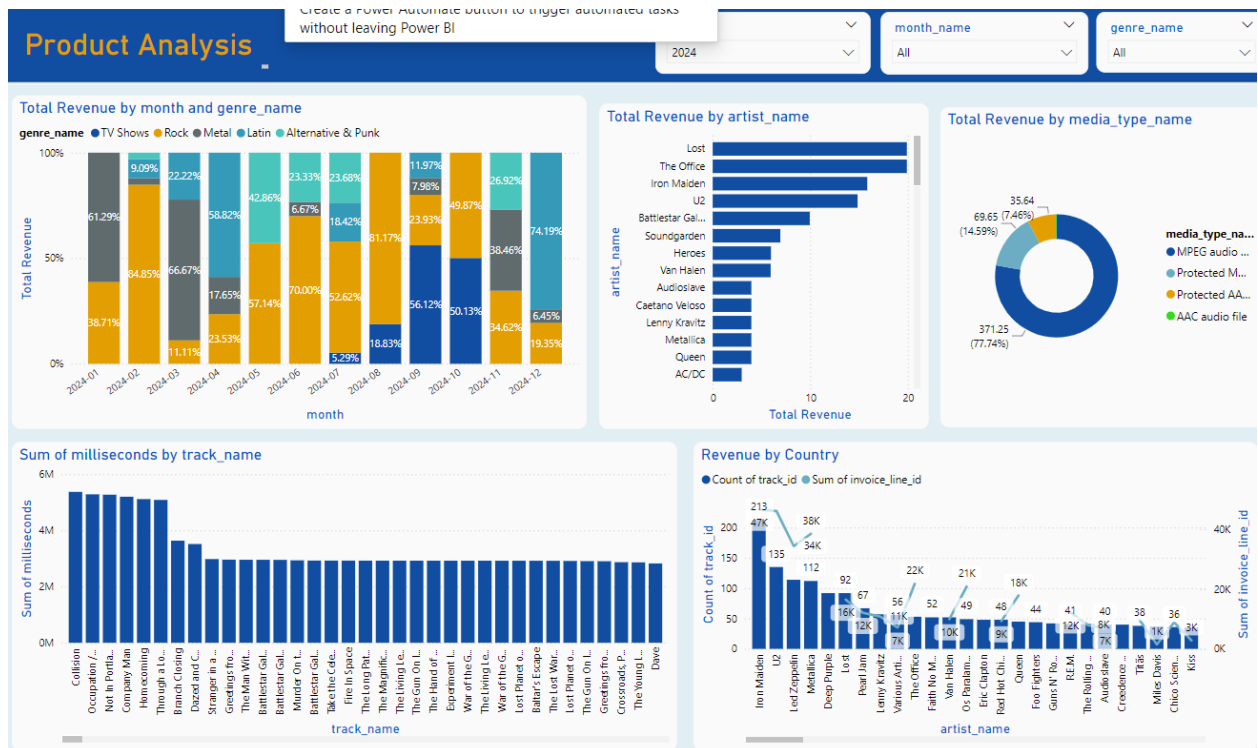


Figure 35: Revenue by each product

The dashboard provides an in-depth analysis of total revenue across various dimensions, including month, genre, artist, media type, and country, highlighting key trends and insights. As mentioned above, from January to July 2024, Rock and Metal genres were the primary revenue drivers, consistently generating steady profits despite their lower price points. When delving deeper into the Rock genre, specifically the artists within this genre, Led Zeppelin and U2 were top contributors, with 114 and 112 songs respectively, demonstrating their strong influence on overall revenue.

A significant revenue surge was observed between August and October 2024, fueled by the introduction of TV Shows, which peaked in September, contributing 56.12% of total revenue. However, as the TV Show trend declined towards the end of the year, there was a resurgence in Alternative & Punk and Latin genres. Despite steady transaction numbers, these genres led to a decrease in overall revenue due to their lower price points.



Figure 36: Revenue by Rock

A deeper dive into the data reveals that Rock songs tend to be shorter than the average track length of 393,599.21 milliseconds. This shorter duration may contribute to higher listening frequency, as customers favor Rock music over other genres. Focusing on producing songs under 400,000 milliseconds could potentially increase customer engagement. This trend is also observed in genres like Metal, Alternative, and Punk.

The majority of revenue came from MPEG audio files, accounting for 77.74%, indicating a clear preference for this format among consumers. Revenue distribution across countries, including the United States, and artists like Iron Maiden and Metallica, suggests a diverse and global customer base.

To further boost revenue, it is essential to increase advertising and promotional efforts for other music genres to attract a broader customer base. Additionally, offering special deals and incentives for loyal customers could help retain their support and encourage continued patronage, thereby enhancing overall company revenue. Overall, the dashboard underscores the significant impact of genre and media type shifts on revenue, with TV Shows playing a

pivotal role in mid-year growth, while the consistent performance of Rock music remains a cornerstone of steady profits.

4.2. Machine Learning

```
customer_data['most_popular_genre'] = customer_data['most_popular_genre'].astype('category').cat.codes
X = customer_data[['total_quantity_purchased', 'total_spend', 'tenure', "unique_genres", "most_popular_genre"]]
y = customer_data['is_churned']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
X_scaled = StandardScaler().fit_transform(X)
# --- Logistic Regression Model Training with Grid Search ---
param_grid = {
    'C': np.logspace(-3, 3, 7),
    'penalty': ['l1', 'l2'],
    'solver': ['liblinear']
}
grid = GridSearchCV(LogisticRegression(random_state = 42), param_grid, cv=5, scoring='f1')
grid.fit(X_train, y_train)
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)
```

Figure 37: The Logistic Regression model

The Logistic Regression model was built to predict customer churn based on transaction data from BigQuery

The procedure involves the following steps. First, the data is preprocessed by converting the date data type, removing duplicate rows, and calculating the total value. Next, the features generated include: the total number of transactions, total spend, tenure (purchase duration), recency (time since last purchase), number of different music genres, and most favorite music genre. The churn label is generated based on the time since the customer's last purchase. Then, the data is normalized and divided into training and testing sets. We trained and tuned the logistic regression model using grid search, and evaluated the Random Forest and Gradient Boosting models. The model is evaluated using accuracy metrics, classification reports, ROC-AUC, and confusion matrix. Finally, the churn prediction result (yes/no) along with the associated features is printed for each customer, along with a pie chart to visualize the distribution of the prediction results.

```

--- Logistic Regression Evaluation Metrics ---
Accuracy: 0.667

```

	precision	recall	f1-score	support
0	0.75	0.50	0.60	6
1	0.62	0.83	0.71	6
accuracy			0.67	12
macro avg	0.69	0.67	0.66	12
weighted avg	0.69	0.67	0.66	12

Figure 38: Evaluation Metrics

The results of the Logistic Regression model evaluation show that the model achieved an overall accuracy of 66.7%. The model performed quite well in classifying the "Yes" class, with a Recall of 83%, which indicates that the model correctly identified most of the instances in this class. The Precision of the "Yes" class was 62%, meaning that 62% of the "Yes" predictions were correct. In contrast, the "No" class had a fairly high Precision (75%), but a low Recall (50%), indicating that the model missed half of the instances that actually belonged to the "No" class. The average F1-Score for both classes was 0.66, reflecting a balanced classification performance at an average level.

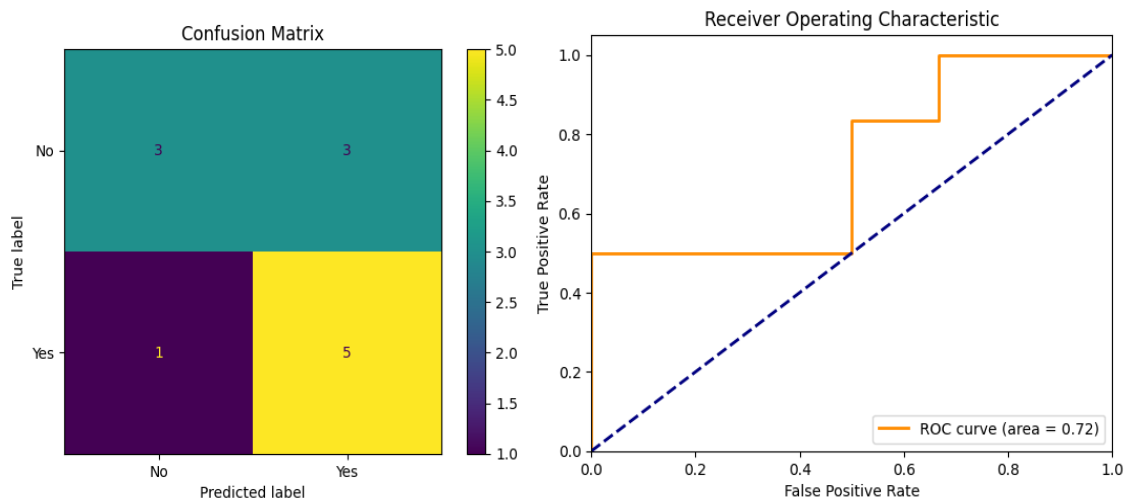


Figure 39: Confusion Matrix

ROC-AUC analysis shows that the model achieved an AUC of 0.722, which means that the discrimination performance between the two classes ("No" and "Yes") is average-good. The ROC curve shows that the model outperforms random prediction but is still far from ideal. The results from the Confusion Matrix show that the model tends to predict the "Yes" class,

with 5 correct predictions and only 1 incorrect prediction from "Yes" to "No". However, for the "No" class, the model only predicted 3 cases correctly and mistook the other 3 cases for "Yes", indicating a significant False Positive error rate.

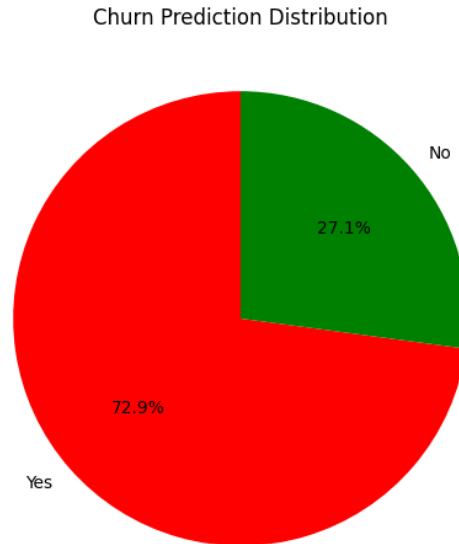


Figure 40: Churn Prediction Distribution

The churn rate is quite high, which is a strong warning sign for the business. There is an urgent need for a customer retention strategy, such as improving service quality and offering incentives or promotions.

Appendix (Codes)

Appendix 1. Composer Dags code

- Extract Dag:

```
37# Database connection function
38def get_postgresql_connection():
39    postgres_conn = {
40        "user": "postgres",
41        "password": "1683",
42        "host": "172.30.112.3",
43        "port": "5432",
44        "database": "postgres"
45    }
46    return create_engine(f"postgresql://{postgres_conn['user']}:{postgres_conn['password']}@"
47        f"{postgres_conn['host']}:{postgres_conn['port']}/{postgres_conn['database']}")
48
49# Fetch table names
50def fetch_table_names():
51    engine = get_postgresql_connection()
52    with engine.connect() as conn:
53        query = """
54        SELECT table_name
55        FROM information_schema.tables
56        WHERE table_schema = 'public' AND table_type = 'BASE TABLE';
57        """
58        table_names = pd.read_sql(query, conn)
59    return table_names['table_name'].tolist()
60
61# Process DataFrame for handling datetime and other types
62def process_dataframe(df, original_types):
63    for col in df.columns:
64        if original_types[col] == 'object':
65            df[col] = df[col].astype(str).replace({'nan': None, 'None': None, "": None})
66        elif original_types[col] == 'datetime64[ns]':
67            df[col] = pd.to_datetime(df[col], errors='coerce').dt.strftime('%Y-%m-%dT%H:%M:%S')
68            df[col] = df[col].where(df[col].notnull(), None)
69
```

```

70# Generate Avro schema
71def get_avro_schema(df, dtype_to_avro, table_name):
72    schema = {
73        "name": table_name,
74        "type": "record",
75        "fields": []
76    }
77
78    for col in df.columns:
79        dtype = str(df[col].dtype)
80        avro_type = dtype_to_avro.get(dtype, 'string') # Default to 'string' if dtype not in mapping
81
82        # Handle nullability
83        if isinstance(avro_type, dict): # For complex types like logicalType
84            if df[col].isnull().any():
85                avro_type = ["null", avro_type]
86            else:
87                if df[col].isnull().any():
88                    avro_type = ["null", avro_type]
89
90        # Special case: Handle datetime64 columns explicitly
91        if pd.api.types.is_datetime64_any_dtype(df[col]):
92            avro_type = ["null", {
93                "type": "long",
94                "logicalType": "timestamp-micros"
95            }]
96
97        # Add field to schema
98        schema["fields"].append({
99            "name": col,
100            "type": avro_type
101        })
102
103    return parse_schema(schema)
104
105# Upload file to GCS with bucket creation
106def upload_to_gcs(bucket_name, source_file_name, destination_blob_name):
107    client = storage.Client()

```

```

108 bucket = client.bucket(bucket_name)
109
110 # Check if the bucket exists
111 if not bucket.exists():
112     logger.info(f"Bucket {bucket_name} does not exist. Creating bucket...")
113     bucket = client.create_bucket(bucket_name)
114     logger.info(f"Bucket {bucket_name} created successfully.")
115
116 # Upload the file
117 blob = bucket.blob(destination_blob_name)
118 blob.upload_from_filename(source_file_name)
119 logger.info(f"Uploaded {source_file_name} to {bucket_name}/{destination_blob_name}")
120
121 # Extract and upload to GCS
122 def extract_and_upload_to_gcs():
123     logger.info("Fetching table names from PostgreSQL...")
124     table_names = fetch_table_names()
125
126     dtype_to_avro = {
127         'object': 'string',
128         'int64': 'long',
129         'float64': 'double',
130         'datetime64[ns]': {
131             "type": "long",
132             "logicalType": "timestamp-micros"
133         },
134         'bool': 'boolean',
135         'category': 'string',
136     }
137
138     for table_name in table_names:
139         logger.info(f"Processing table {table_name}")
140         engine = get_postgresql_connection()
141         with engine.connect() as conn:
142             query = f"SELECT * FROM \"{table_name}\""
143             df = pd.read_sql(query, conn)
144
145         original_types = df.dtypes

```



```

146     process_dataframe(df, original_types)
147
148     # Generate Avro schema
149     parsed_schema = get_avro_schema(df, dtype_to_avro, table_name)
150
151     # Save Avro file locally
152     avro_file_path = f'tmp/{table_name}.avro'
153     logger.info(f"Saving Avro file for table {table_name} to {avro_file_path}")
154     with open(avro_file_path, 'wb') as out:
155         records = df.to_dict(orient='records')
156         for record in records:
157             for key, value in record.items():
158                 if pd.isna(value):
159                     record[key] = None
160         writer(out, parsed_schema, records)
161
162     # Upload Avro file to GCS
163     logger.info(f"Uploading Avro file for table {table_name} to GCS...")
164     upload_to_gcs(BUCKET_NAME, avro_file_path, f'{table_name}.avro')
165
166     # Remove local file
167     os.remove(avro_file_path)
168     logger.info(f"Removed local file {avro_file_path}")

```

- Extract Dag:

```

15def list_gcs_files(bucket_name):
16     """List all file names in a Google Cloud Storage bucket without extensions."""
17     storage_client = storage.Client()
18     blobs = storage_client.list_blobs(bucket_name)
19     # Remove the '.avro' extension from each file name
20     return [blob.name.rsplit('.', 1)[0] for blob in blobs if blob.name.endswith('.avro')]
21
22
23def create_bigquery_dataset_if_not_exists(project_id, dataset_name):
24     """Create a BigQuery dataset if it does not exist."""
25     client = bigquery.Client()
26     dataset_id = f'{project_id}.{dataset_name}'

```

```

27
28 try:
29     client.get_dataset(dataset_id) # Check if dataset exists
30     print(f"Dataset {dataset_id} already exists.")
31 except NotFound:
32     # Create the dataset
33     dataset = bigquery.Dataset(dataset_id)
34     dataset.location = "US" # Specify your dataset location
35     client.create_dataset(dataset, exists_ok=True)
36     print(f"Created dataset {dataset_id}.")
37
38 # Define default arguments for the DAG
39 default_args = {
40     'owner': 'airflow',
41     'depends_on_past': False,
42     'retries': 1,
43 }
44
45 # Initialize the DAG
46 with DAG(
47     dag_id='gcs_to_bigquery_avro',
48     default_args=default_args,
49     start_date=datetime(2024, 12, 6), # Replace with your desired start date
50     schedule_interval=None, # Define the schedule (e.g., daily, hourly)
51     catchup=False,
52 ) as dag:
53
54     # Task to ensure BigQuery dataset exists
55     ensure_dataset = PythonOperator(
56         task_id='ensure_bigquery_dataset',
57         python_callable=create_bigquery_dataset_if_not_exists,
58         op_kwargs={
59             'project_id': PROJECT_ID,
60             'dataset_name': DATASET_NAME,
61         },
62     )
63
64     list_files = list_gcs_files(BUCKET_NAME)

```

```

65 previous_task = ensure_dataset
66
67 # Loop through files and create tasks to load each file into BigQuery
68 for table_name in list_files:
69     load_avro_to_bigquery = GCSToBigQueryOperator(
70         task_id=f"load_{table_name}_to_bigquery",
71         bucket=BUCKET_NAME,
72         source_objects=[f"{table_name}.avro"],
73         destination_project_dataset_table=f"{PROJECT_ID}.{DATASET_NAME}.{table_name}",
74         source_format='AVRO',
75         write_disposition='WRITE_TRUNCATE',
76         autodetect=True,
77     )
78
79 # Ensure this task runs after the dataset creation check
80 previous_task >> load_avro_to_bigquery
81 previous_task = load_avro_to_bigquery

```

Contribution

Name	StudentID	Contribution
Lê Tuấn Sơn	21070340	100%
Lưu Hà Linh	21070757	100%
Hoàng Anh Đức	21070190	100%
Phạm Thị Mỹ Dung	21070509	100%
Phạm Thị Lan Anh	21070137	100%